Chapter 23: Minimum Spanning Trees

1) Given an acyclic, undirected graph $G = (V, E)$ and a weight function $w: E \to R$ we want to determine a spanning tree (called so because it spans the graph in the sense that it connects all of the vertices) with the minimum weight.

→ The phrase "minimum spanning tree" is a shortened form of the phrase "minimum-weight spanning tree". We cannot minimize the no. of edges in the spanning tree, since all spanning trees have exactly $|V| - 1$ edges.

→ Minimum spanning tree for a graph may not be unique. There may be different ones that all achieve the same minimum weight.

2) Generic algorithm for growing a minimum spanning tree:

The greedy strategies used by Kruskal and Prim is captured by the following generic algorithm that grows the minimum spanning tree one edge at a time.

GENERIC_MST(G,W) ▷ w = weight function

```
1  A ← ϕ
2  While A does not form a spanning tree
3        do find an edge (u,v) that is safe for A
4           A ← A ∪ {(u,v)}
5  return A
```

The algorithm manages a set of edges A, maintaining the following loop invariant:

"Prior to each iteration, A is a subset of some minimum spanning tree".

During each iteration we determine an edge (u,v) that can be added to A without violating this invariant, in the sense that A ∪ {(u,v)} is also a subset of a minimum spanning tree. We call such an edge a "safe edge" for A, since it can be added to A while maintaining the invariant.

Initialization: After initializing A with an empty set, it trivially satisfies the loop invariant.

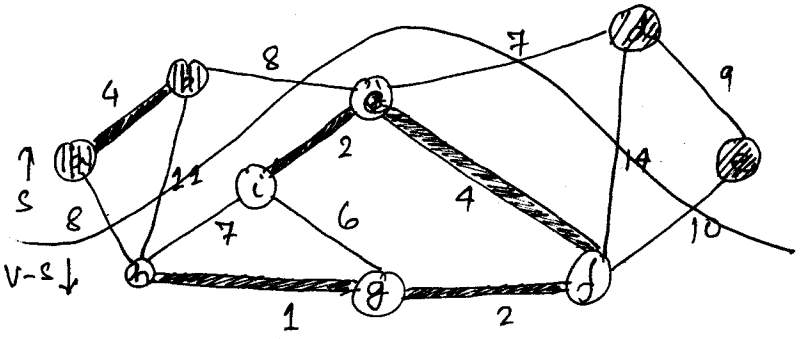
Maintenance: The loops maintains the invariant by adding only safe edges.

Termination: Since A started out as an empty set and during each iteration of the loop we add an edge that belongs to a minimum spanning tree and since termination occurs only when A is a ~~minimum~~ spanning tree, at termination the set a must be a minimum spanning tree.

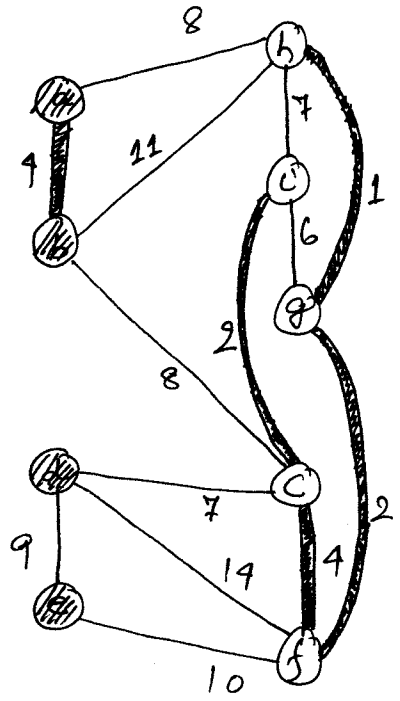
→ A cut (S, V-S) of an undirected graph G = (V, E) is a partition of V. An edge is said to cross a cut (S, V-S) if one of its edges is in S and the other is in V-S. A cut is said to respect a set A of edges if no edge in A crosses the cut. Of all edges crossing a cut the edge with the minimum weight is called a "light edge".

More generally, we say that an edge is a light edge satisfying a given property if its weight is a minimum of all edges satisfying the property.

eg:-



≡

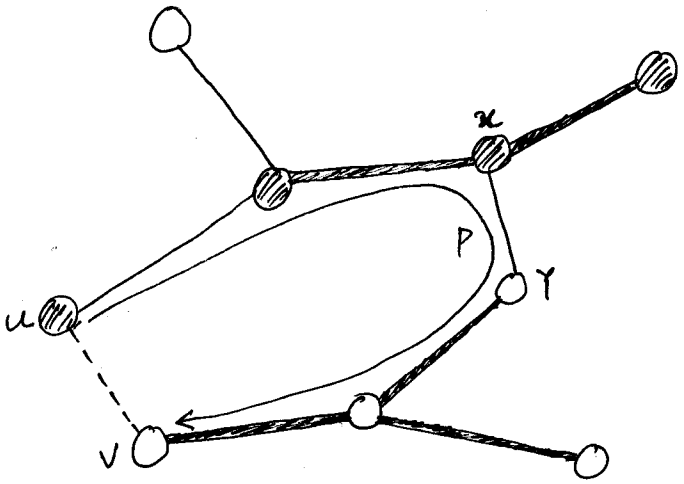


The vertices in S are shown in black and the vertices in V-S are shown in white. The edges of A are shown shaded.

Theorem

Let $G = (V, E)$ be a connected, undirected graph. Let A be a subset of E that is included in some minimum spanning tree for G. Let $(S, V-S)$ be a cut that respects A. Let (u, v) be a light edge crossing $(S, V-S)$. Then, edge (u, v) is safe for A.

Proof: Let T be a minimum spanning tree of G that includes A. We assume that T does not include the light edge (u, v), since if it does, we are done. We shall construct another mst T' that includes $A \cup \{(u, v)\}$ by using a cut-and-paste technique, thereby showing that (u, v) is a safe edge for A.



All edges in set T are shown. Other edges of G have not been shown. Edges in A have been shaded.

The edge (u,v) forms a cycle with the edges on the path p from u to v in T. Since (u,v) is a light edge, u and v must be on the opposite sides of the cut (S, V-S). Hence, there must be at least one edge in T on the path p that also crosses the cut. Let (x,y) be such edge. (x,y) cannot be in A since the cut respects A.

Let $T' = T - \{(x,y)\} \cup \{(u,v)\}$

Since, (u,v) is a light edge $w(u,v) \leq w(x,y)$

$$w(T') = w(T) - w(x,y) + w(u,v)$$

$$\leq w(T) .$$

But T is a minimum spanning tree, so $w(T) \leq w(T')$; thus T' is also a set that achieves the same wt.

→ The set A must always be acyclic, otherwise the set that includes A would contain a cycle. At any point during the execution of the algorithm the graph $G_A = (V, A)$ is a forest (that includes every vertex in G, but only those edges which are in A). Each connected component of G_A must be

a tree. Some of the trees may contain just one vertex, which is the case when the algorithm begins: Set A is empty and the forest contains $|V|$ trees, one for each vertex.

Any safe edge (u,v) for A connects distinct components of G_A, since $A \cup \{(u,v)\}$ must be acyclic.

The loop of GENERIC_MST is executed $|V|-1$ times as each of the $|V|-1$ edges of a minimum spanning tree is successively determined. Initially when $A = \phi$, there are $|V|$ trees in G_A, and each iteration reduces this number by 1. The algorithm terminates when the forest contains only a single tree.

3) Kruskal's algorithm

→ Kruskal's algorithm is based directly on the general minimum spanning tree algorithm. In Kruskal's algorithm the set A is a forest. The safe edge that is added to A is always a least-weight edge in the graph that connects 2 distinct components.

→ The algorithm starts as a set of $|V|$ trees, where every tree is a single vertex. It sorts the $|E|$ edges of the graph in a non-decreasing order and considers the edges in this order (each edge is considered At every step the edge under consideration has minimum wt. of all the just once). If the edge under consideration joins unconsidered edges 2 distinct trees the edge is included in the forest A, otherwise ignored.

→ The implementation of Kruskal's algorithm uses a disjoint-set data structure to maintain

several disjoint set of elements. Each set contains the vertices in a single tree of the current forest. The operation FIND_SET(u) returns a representative element from the set that contains u. The UNION procedure combines 2 sets thus merging their trees into a single tree.

MST_KRUSKAL (G,W)

```
1   A ← φ
2   for each vertex v ∈ V[G]
3       do MAKE_SET (v)
4   sort the edges of E into non-decreasing order
    by weight W
5   for every edge (u,v) ∈ E , taken in non-decreasing
    order by weight
6       do if FIND_SET(u) ≠ FIND_SET(v)
7           then A ← A ∪ {(u,v)}
8                UNION (u,v)
9   return A
```

→ The running time of Kruskal's algorithm depends upon the implementation of the disjoint-set data structure. We shall assume the disjoint-set-forest implementation with the union-by-rank and path-compression heuristics, since it is the asymptotically

fastest implementation known.

Initialising A with empty set takes O(1) time and sorting the edges of E can be performed in O(E lg E). The for loop performs O(E) FIND_SET and UNION operations on the disjoint-set-forest. These along with the |V| MAKE_SET operations, take a total time of $O((V+E)\alpha(V))$ where α is the very slowly growing function defined in sec. 21.4. Since G is assumed to be connected, we have $|E| \geq |V|-1$ and so the disjoint-set operations take $O(E\alpha(V))$ time. Since $\alpha(V) = O(\lg V) = O(\lg E)$, the total running time of Kruskal algorithm is $O(E \lg E)$. Observing that $|E| < |V|^2$ we have $\lg |E| = O(\lg V)$ and hence Kruskal's algorithm can be restated as $O(E \lg V)$.

4) Prim's algorithm

In Prim's algorithm the set A forms a single tree. The safe edge added to A is always a least weight edge connecting the tree to a vertex not in the tree. The tree starts with an arbitrary root-vertex r and grows until the tree spans all the vertices in V

The MST_PRIM procedure keeps all those vertices which are not in the tree in a min-priority queue Q based on a key field. For a vertex v, Key[v] is the minimum weight of any edge that connects v to some vertex in the tree. Initially $Key[v] = \infty \ \forall v \neq r$, since none of the vertices is

connected to the tree. $\pi[v]$ contains the parent of v in the tree.

MST_PRIM (G, W, r)

```
1   for each u ∈ V[G]
2       do Key[u] ← ∞
3          π[u] ← NIL
4   Key[r] ← 0
5   Q ← V[G]
6   while Q ≠ ϕ
7       do u ← EXTRACT_MIN(Q)
8          for every v ∈ Adj[u]
9              do if v ∈ Q and w(u,v) < Key[v]
10                 then π[v] ← u
11                      Key[v] ← w(u,v)
```

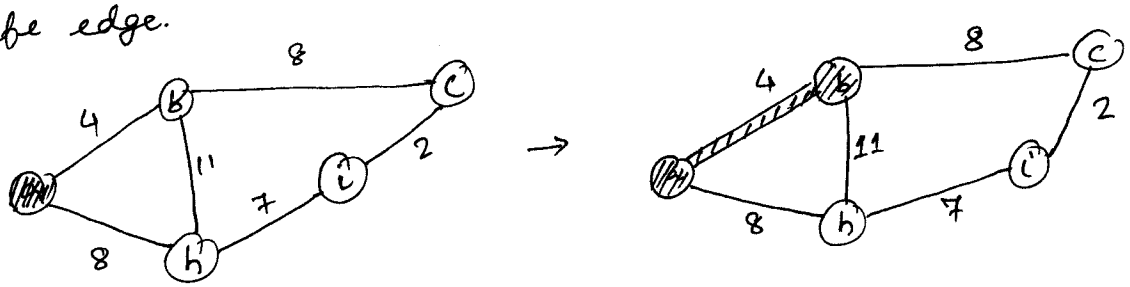
→ During the execution of the algorithm, MST_PRIM implicitly maintains the set A as:—

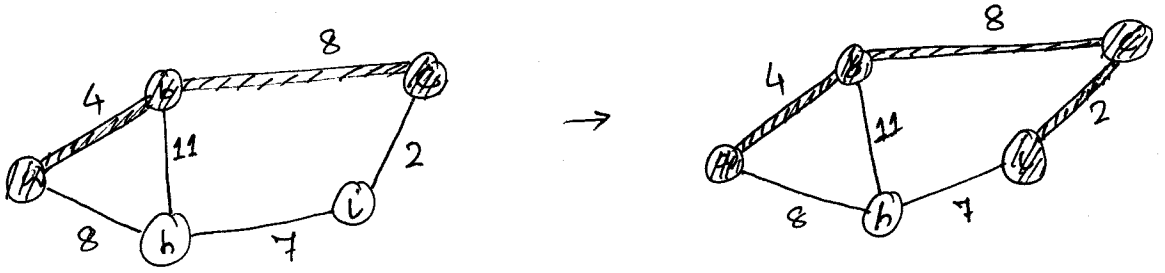
$A = \{ (\pi(v), v) : v \in V - \{r\} - Q \}$ which at termination becomes:-

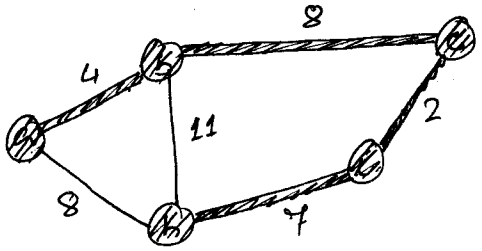
$A = \{ (\pi(v), v) : v \in V - \{r\} \}$

→ Extraction of the vertex with the minimum Key in line 7, implicitly adds the edge $(\pi[u], u)$ to A.

At every step the vertices in the tree (i.e. $V-Q$) determine a cut of the graph $(V-Q, Q)$ and the edge $(\pi[u], u)$ is a light edge that crosses the cut. Thus $(\pi[u], u)$ is a safe edge.







black vertices = $V-Q$
White vertices = Q
edges in A = Shaded

→ The performance of Prim's algorithm depends upon how we implement the min-priority queue Q. If Q is implemented as a binary min-heap we can use the BUILD-MIN-HEAP procedure to perform the initialization in lines 1-5 in $O(V)$ time.

The body of while loop is executed $|V|$ times, and since each EXTRACT-MIN operation takes $O(\lg V)$ time the total time for all calls to EXTRACT-MIN is $O(V \lg V)$.

The for loop is executed $O(E)$ times altogether, since the sum of the lengths of the adjacency lists is $2|E|$. Within the for loop, the test for

membership in Q can be implemented in constant time by keeping a bit for every vertex that tells whether or not it is in Q.
The assignment in line 11 involves a DECREASE-KEY operation on the min-heap, which can be implemented in a binary min-heap in $O(\lg V)$ time.

Thus, the total time for Prim's algorithm is $O(V \lg V + E \lg V) = O(E \lg V)$, which is asymptotically the same as for our implementation of Kruskal's algorithm.

→ The asymptotic running time of Prim's algorithm can be improved by using Fibonacci heaps. If $|V|$ elements are organized into a Fibonacci heap, we can perform an EXTRACT-MIN operation in $O(\lg V)$ amortized time and a DECREASE-KEY operation in $O(1)$ amortized time. Thus, the run time of PRIM's algorithm using Fibonacci heaps is $O(V \lg V + E)$.